

Software Design Document

— TJ Drape, Afonso Azevedo, Roman Zalewski, John Markham —

GitHub Link: https://github.com/t-drape/SE300_Secure_Notes_App

1. Introduction and Purpose

The System Under Development (SUD) is a local command-line secure notes application that provides users with a privacy-focused way to create, manage, and analyze personal notes. Users can create new notes, read previously saved notes, append additional content to existing notes, and delete notes they no longer need. The system also includes AI-assisted features that allow users to generate summaries of their notes and extract important keywords. These features help users quickly identify the most important information in a note.

Protecting the confidentiality of user data is one of the primary goals of the SUD. To support this goal, the system encrypts all note content using AES-256 encryption before saving it to storage. The system derives encryption keys from a user-provided password using the PBKDF2 (Password-Based Key Derivation Function 2) algorithm. This approach ensures that only users who know the correct password can access the stored notes.

The application runs entirely on the Host Computer (HC) and provides all functionality through a menu-driven Command Line Interface (CLI). The system operates fully offline and does not rely on any cloud services or external infrastructure. It stores encrypted notes locally in a SQLite3 database, which allows the system to manage note storage efficiently while maintaining full local control over the data.

The system uses a modular architecture that separates major responsibilities into dedicated components. These components handle user interaction, local data management, encryption and authentication, and AI-based note analysis. This design improves maintainability, keeps the system organized, and makes it easier to extend individual components in the future.

1.1 Acronyms, Terms and References

Note: For Class Diagrams and Pseudocode, we utilize UML convention of + and – to denote public and private fields and methods, respectively.

Acronym	Meaning
HDD	High-Level Design
LLD	Low-Level Design

SDD	Software Design Document
SUD	System Under Development
HC	Host Computer
SRD	Software Requirements Document
TSC	Terminal Start Command
PEC	Program Exit Command (^C)
CRUD	Create, Read, Update, Delete (Programming Actions)

Term	Definition
Temporary Files	Files that the system generates and passes between internal functions during execution but do not permanently store on the Host Computer or in the database. The system uses these files only during runtime.
Encrypted Note	A note whose content the system has transformed using AES-256 encryption before storing it in the database.
Decrypted Note	The plaintext version of a note that the system reconstructs in memory after decrypting the stored encrypted data.
Note Analysis	The process of analyzing note content to generate summaries or identify important keywords using the system's AI-assisted processing module.

Reference	Source
<i>SRD for AI-powered Secure Notes Application</i>	SRD.pdf (accessible on GitHub)

2 Design Considerations

2.1 Assumptions and Dependencies

The design of the SUD relies on several assumptions about the execution environment and system behavior.

- The SUD runs entirely on the HC and does not rely on any external servers or network services. All functionality is expected to operate offline.
- The application executes a TSC through a CLI in a terminal environment. The system assumes the user has access to a compatible terminal that can execute the program.
- The system stores encrypted notes locally using a SQLite3 database. The design assumes the host environment supports the SQLite3 library required for local data persistence.
- The system assumes that users provide a valid password when accessing encrypted notes. This password is used to derive encryption keys via PBKDF2.
- The AI-assisted note analysis module operates on plaintext note content that exists temporarily in memory after decryption. The system assumes that sufficient system memory is available to process notes up to the maximum size specified in the SRD.

Additional changes to system requirements or operating environments may occur in the future. Therefore, the design separates core responsibilities such as data management, encryption, and analysis into distinct modules to allow future modifications with minimal impact on the overall system.

2.2 Design Constraints

As currently specified in SRD, there are no constraints placed on the SUD's design, and none are levied herein. That said, the design has been constructed such that the potential upgrades mentioned in section 2.1 should be easily accommodated.

2.3 Goals and Guidelines

The SUD has been designed with modifiability, maintainability, security, and performance in mind (see Section 2.1 for potential changes). Consequently, sound software design principles have been emphasized, especially:

- Modularity and information hiding: The design separates concerns to enable independent modification and testing of encryption, note management, user interface, and AI processing functionality.
- Loose coupling between modules: Dependencies are minimized to prevent cascading effects throughout the system.
- Loose coupling within modules: Member functions rely on parameters to pass data rather than global variables, ensuring variables and data exist only where needed and used.
- High cohesion within modules: Each module has a single, well-defined responsibility.
- Partitioning volatility: Components likely to change are isolated from stable components to accommodate future requirements with minimal impact.

- Security-focused design: The architecture enforces AES-256 encryption, PBKDF2 key derivation, and ensures plaintext content never persists outside application memory.

2.4 Development Methods

The team first created the Software Requirements Document (SRD) to define the system's functional and non-functional requirements. This Software Design Document (SDD) builds on those requirements by describing how the system is structured and how its components interact.

To develop the SUD, the team follows an iterative, object-oriented approach combined with modular design principles. The system is divided into independent components, each with a clear responsibility. For example, separate modules handle data storage, encryption and decryption, AI-based note analysis, and user interaction through the CLI. This modular structure allows team members to work on different parts of the system remotely while keeping the overall architecture organized and maintainable.

The team also follows a test-driven development (TDD) process. Before integrating components into the full system, the team implements and tests individual modules to verify that they behave as expected. This approach helps identify issues early in development and ensures that each component functions correctly before it interacts with other parts of the system.

3 High-Level Design (HLD)

This section presents the high-level architectural decomposition of the Secure Notes Application (SUD).

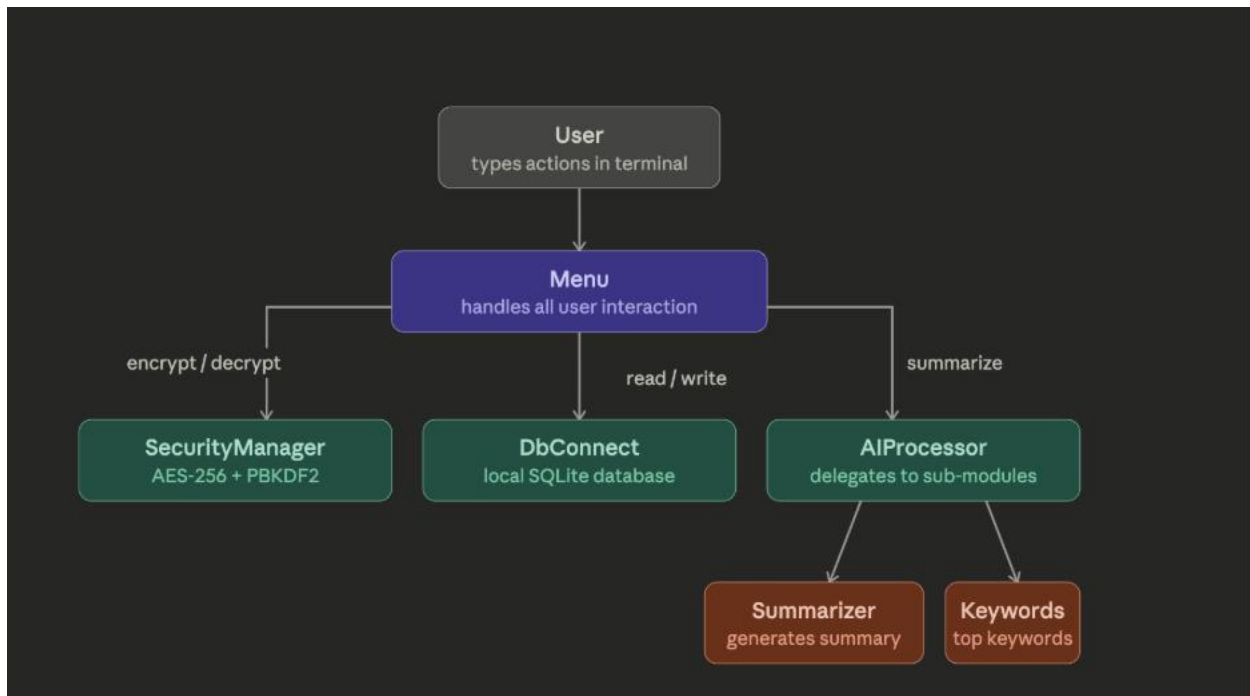


Figure 1: Module Interaction Diagram

Figure 1 shows a top-level view of the SUD's architecture and the relationships between its primary modules. The design centers on a menu-driven CLI that coordinates local data management, encryption services, and AI-assisted note analysis.

Figure 1 Legend and Diagram Explanation

- The **gray box (User)** represents the external actor interacting with the system. The user types commands into the terminal to create, read, append, delete, or analyze notes.
- The **blue box (Menu)** represents the central controller of the application. The Menu module manages all user interaction through the command-line interface (CLI) and determines which system action should be executed based on the user's input.
- The **green boxes** represent the system's core service modules:
 - **SecurityManager** handles encryption and decryption using AES-256 encryption and PBKDF2 key derivation.
 - **DbConnect** manages persistent storage of encrypted notes using the local SQLite3 database.
 - **AIProcessor** coordinates AI-assisted analysis and delegates analysis tasks to specialized AI submodules.
- The **orange boxes** represent the AI analysis submodules:
 - **Summarizer**, which generates extractive summaries from note content.
 - **KeywordAnalyzer**, which identifies significant keywords within the note text.
- **Solid arrows** represent control flow and communication between modules. They show how requests and data move between components during program execution.

- The **labels on the arrows** describe the type of operation being performed. For example, actions such as display, append, and save involve communication between the Menu and the system's data and security modules, while summarize and keyword analysis requests are delegated to the AIProcessor and its AI submodules.

3.1.1 Menu [SDD_HLD_01_MENU]

This object has three main responsibilities:

1. Receive user commands through the CLI and determine which system action to perform.
Trace Tag: SDD_HLD_MENU_001
2. Coordinate interactions between the user and the system's major modules, including data management, security services, and AI-assisted note analysis.
Trace Tag: SDD_HLD_MENU_002
3. Present results, prompts, and feedback messages to the user.
Trace Tag: SDD_HLD_MENU_003

3.1.2 SecurityManager [SDD_HLD_02_SEC]

This object has three main responsibilities:

1. Derive encryption keys from a user-provided password using PBKDF2, ensuring entered password passes pattern validation.
Trace Tag: SDD_HLD_SEC_001
2. Encrypt note content before the system stores it in the local database while gracefully handling errors associated with encryption.
Trace Tag: SDD_HLD_SEC_002
3. Decrypt note content in memory when the user requests to view or analyze a stored note and gracefully handling errors associated with decryption.
Trace Tag: SDD_HLD_SEC_003

3.1.3 DbConnect [SDD_HLD_03_DB]

This object has one main responsibility:

1. Initialize and manage access to the local, offline-operational SQLite3 database while performing efficient CRUD operations on encrypted notes.
Trace Tag: SDD_HLD_DB_001

3.1.4 AI Class

3.1.4.1 AIProcessor [SDD_HLD_04_01_AI]

This object has one main responsibility:

1. Receive note-analysis requests from the Menu component and delegate them to the appropriate AI sub-module, while gracefully handling errors with file size.

Trace Tag: SDD_HLD_AI_001

Note: The current design separates AI orchestration from the implementation of specific analysis algorithms.

3.1.4.2 Summarizer [SDD_HLD_04_02_SUM]

This object has one main responsibility:

1. Generate extractive summaries of decrypted note content by ranking sentences according to their informational importance and return a concise summary output.

Trace Tag: SDD_HLD_SUM_001

Note: The current summarization design uses a rule-based NLP approach that preprocesses, analyzes word frequencies, and ranks sentences.

3.1.4.3 KeywordAnalyzer [SDD_HLD_04_03_KEY]

This object has one main responsibility:

1. Extract significant keywords from decrypted note content by preprocessing the text and analyzing word frequency and return them as a ranked list.

Trace Tag: SDD_HLD_KEY_001

Note:

- The current keyword analysis design uses a rule-based NLP approach that preprocesses note text, removes common stopwords, and ranks remaining words according to frequency.
- This design allows future upgrades to more advanced keyword extraction methods without affecting the rest of the system.

The modules shown in Figure 1 work together to support the core functionality of the Secure Notes application. The user interacts with the system through the menu-driven CLI, which serves as the central coordinator for all system operations. When the user performs an action such as creating, reading, appending, deleting, or analyzing a note, the Menu module directs the request to the appropriate system component.

For operations involving note storage or retrieval, the Menu communicates with the DbConnect module, which manages the SQLite3 database that stores encrypted notes. When encryption or decryption is required, the system uses the SecurityManager module to ensure that note content remains protected while stored and is only decrypted in memory during runtime operations.

When the user requests AI-based analysis, the Menu sends the decrypted note content to the AIProcessor. The AIProcessor then delegates the task to the appropriate analysis module. The Summarizer generates extractive summaries by identifying the most important sentences in the note, while the KeywordAnalyzer identifies significant keywords within the text.

This modular interaction ensures that responsibilities remain clearly separated across the system. The Menu coordinates user requests, DbConnect manages persistent storage, SecurityManager enforces data protection, and the AI modules perform content analysis.

Together, these components provide a secure, organized, and maintainable architecture for the SUD.

3.2 Top-Level Functional View

The SUD accepts user actions through a menu-driven CLI, delegates security operations to a dedicated Security Manager, and saves encrypted data to the local file system. The system has one primary actor (the User) and one secondary actor (the local file system, referred to as Database).

The six top-level use cases derived from the SRD and the use-case diagram are:

- Create Note - the user creates a new note, and the system encrypts and saves it.
- Display Note (Read) - the user selects a saved note, the system decrypts and displays it in memory only.
- Change Note (Append) - the user selects a saved note to make changes to, the system decrypts, appends new content, re-encrypts, and saves.
- Summarize Note - the user selects a note; the system decrypts it, passes plaintext to the AI module, and displays the plaintext summary.
- Delete Note - the user selects a note for deletion, the system requests confirmation, then removes the file permanently.
- Save Note - a private operation that follows the Create/Append note that writes the encrypted note to the local file system.

4 Low-Level Design (LLD)

This section describes the internal structure and behavior of the system components introduced in the High-Level Design. While the HLD presents the overall architecture and relationships between major modules, the Low-Level Design explains how those modules are implemented in detail.

The LLD includes class-level diagrams and pseudocode that define the internal data structures, methods, and interactions required to implement the Secure Notes Application. Each module described in Section 3 is broken down into its corresponding classes and functions.

4.1 Project Flow

This section describes how data runs through the system and the main actors within the SUD. We use common UML diagrams to help visualize the connection between classes, the functions that make these connections, and how the user and DB start these processes.

4.1.1 Use Case Diagram

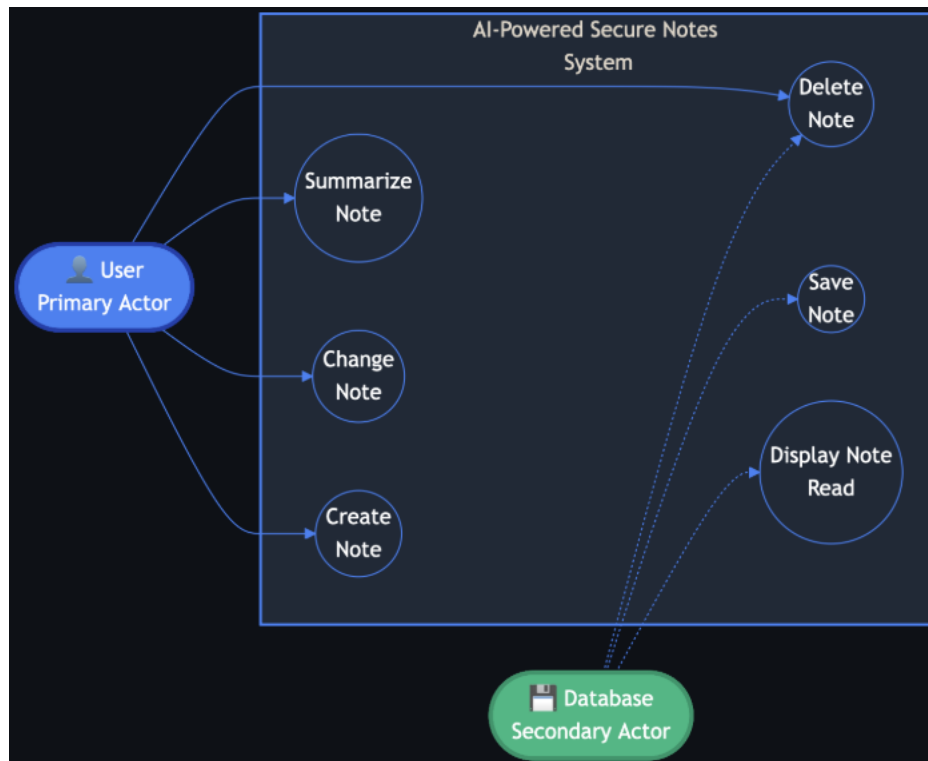


Figure 2: Use Case Diagram

Figure 2 illustrates the primary interactions between the user and the Secure Notes Application. The User acts as the primary actor and interacts with the system through the command-line interface to create, read, modify, delete, or summarize notes.

The Database serves as a secondary actor that provides persistent storage for encrypted notes. Operations such as saving, retrieving, and deleting notes require communication between the system and the database.

The use cases shown in the diagram correspond to the main functional capabilities defined in the Software Requirements Document (SRD).

4.1.2 Sequence Diagram



Figure 3: Use Case Diagram

Figure 3 shows how the components of the SUD are related. The lines represent function calls and show how data flows through the system. Each arrow acts as a bi-directional communication channel, a way for the one module to talk to another, receive data and then act on the manipulated return data. Each module passes data to the next through parameters, enhancing security by minimizing exposure of global variables. This decoupling highlights the object-oriented approach, as each module completes a single goal, and then transfers the final product to another component to be processed further.

4.2 Components

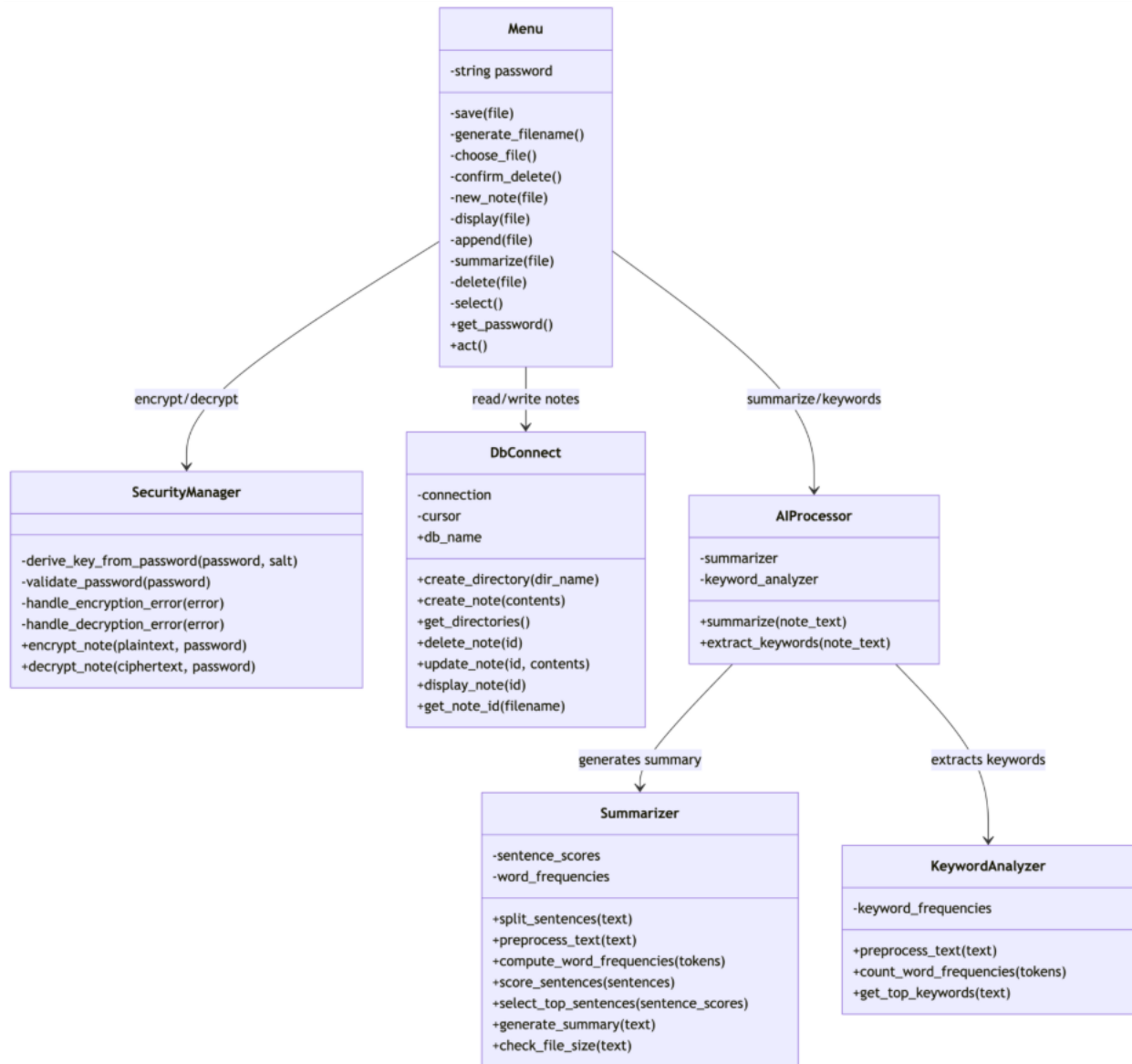


Figure 4: Full UML Class Diagram

Figure 4 shows a complete UML class diagram of our software design, with the arrows showing when and how data flows through the SUD. Focusing on modular decomposition, each box represents a unique component. These components only interact through member function calls and mainly use private data attributes. This decoupling enhances security, ensuring no data such as a plaintext note or a password is ever saved or shared outside the scope of the current function.

4.2.1 Menu Class



Figure 5: Menu Class

Figure 5 shows the structure of the Menu class. The Menu class acts as the central controller of the Secure Notes Application. It receives user commands through the command-line interface (CLI) and coordinates the execution of system operations such as creating, displaying, modifying, summarizing, and deleting notes.

The Menu class stores the user's password during runtime and uses it when interacting with the encryption and decryption services provided by the SecurityManager module. Based on the user's selected action, the Menu class calls the appropriate system components, including the database manager and the AI processing modules.

The component contains one private attribute, password, which stores the user-entered password. This password is used to derive encryption keys for securing note data.

The Menu class also defines several methods that implement the main system actions. These methods include generating filenames for notes, selecting notes from storage, saving notes, displaying note content, appending new content to existing notes, generating summaries, and deleting notes. Additional helper methods manage user input and coordinate the execution of selected actions.

4.2.1.1 Menu Pseudocode:

Note:

Any DB actions are performed by the DB component

Any Encryption or Decryption actions are performed by the SecurityManager component

-save(string file): SDD_TT_3_001

file = filename,

If file content exists,

Encrypt file contents from writing window, (passing password)

Save encrypted file content to DB under filename

-generate_filename(): SDD_TT_3_002

Find current date,

Map to appropriate filename, like 03_10_26 for March 10th, 2026

Return filename

-choose_file(): SDD_TT_3_003

Print message for user input,

If file exists,

Return filename

Else,

Return to start of program

-confirm_delete(): SDD_TT_3_004

Print message for user input,

If user input = confirmation,

Return true

Else,

Return false

-new_note(): SDD_TT_3_005

Generate filename,

If written content,

Save file under filename (call save function)

-display(string file): SDD_TT_3_006

file = filename,

Get file from DB,
Decrypt file contents and save them to a temporary file, (passing password)
Open temporary file in read-only mode

-append(string file): SDD_TT_3_007
 file = filename,
 Get file of filename from DB,
 Write decrypted file contents to temporary file, (pass password)
 Open temporary file in append mode in writing window,
 Save file (call the Save function)

-summarize(string file): SDD_TT_3_008
 file = filename,
 Get file of filename from DB,
 Decrypt file contents and write to a temporary file, (pass password)
 Call the AI class summary function and pass the decrypted temporary file,
 Get return value from AI summary function,
 Display returned file contents

-delete(string file): SDD_TT_3_009
 file = filename,
 Call DB delete function, passing filename parameter

-select(): SDD_TT_3_010
 Print message for user input for chosen action,
 Return input

+get_password(): SDD_TT_3_011
 Print message for user input,
 If user input exists and matches requirements, (eight characters)
 Save to password variable
 Else,
 Call get_password

+act(): SDD_TT_3_012
 Call select function,
 Map input to action,
 1) New Note:
 Call new_note function
 2) Display Note:
 Call choose_file function,
 Pass file to display function
 3) Change Note:
 Call choose_file function,
 Pass file to append function

- 4) Summarize Note:
 - Call choose_file function,
 - Pass file to summarize function
- 5) Delete Note:
 - Call choose_file function,
 - Pass file to delete function
- 6) Default:
 - Restart program
 - Print Action Type

4.2.2 SecurityManager Class

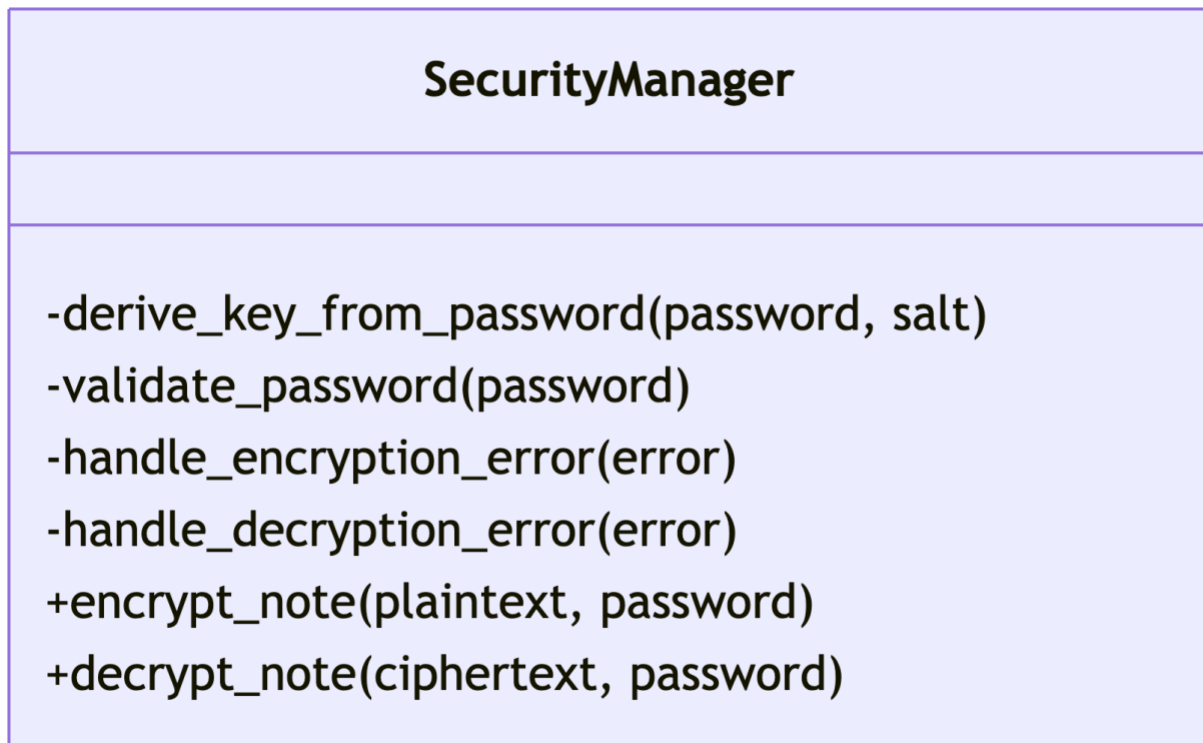


Figure 6: SecurityManager Class

The SecurityManager class is a stateless utility class that is responsible for all encryption and decryption for the SUD. It contains two public methods, `encrypt_note` and `decrypt_note` which are called exclusively by the Menu class. All other methods are private internal helpers that Menu never interacts with directly.

When the Menu calls `encrypt_note`, the method first validates that the password meets the minimum length requirement, then generates a random salt and derives an AES-256 key using PBKDF2 with the password and salt, encrypts the plaintext, and returns the salt concatenated with the ciphertext. This concatenated string is what gets stored in the database by DbConnect. The key and salt exist only as local variables for the duration of the function operations and are never stored anywhere that persists.

When Menu calls `decrypt_note`, the method validates the password, extracts the salt from the front of the ciphertext bundle, re-derives the same key using PBKDF2 with the same password and salt, and decrypts the ciphertext. The returned plaintext is held in memory by Menu only for as long as it is needed and again, is never written to disk for privacy assurances.

The class never stores the password, key, or salt between method calls. All sensitive data is limited to individual function calls and discarded after use. This design makes sure that plaintext and cryptographic material never persist outside of the active function call, satisfying SRD::T_05, SRD::T_47, SRD::T_48, and SRD::T_50.

Error handling is managed by two private methods- `handle_encryption_error` and `handle_decryption_error`- which are called internally when the validation fails, thus satisfying SRD::T_51 and SRD::T_52.

4.2.2.1 SecurityManager Pseudocode:

-derive_key_from_password(password, salt): SDD_TT_4_001 ::

[SRD::T_15, SRD::T_46]

 Use PBKDF2 to derive key from password and salt,

 Return key

-validate_password(password): SDD_TT_4_002 :: [SRD::T_31]

 If password length < 8,

 Return false

 Else,

 Return true

-handle_encryption_error(error): SDD_TT_4_003 :: [SRD::T_51]

 Print encryption error message to user,

 Return to caller

-handle_decryption_error(error): SDD_TT_4_004 :: [SRD::T_52]

 Print decryption error message to user,

 Return to caller

+encrypt_note(plaintext, password): SDD_TT_4_005

:: [SRD::T_15, SRD::T_28, SRD::T_35,

 SRD::T_46, SRD::T_47, SRD::T_49]

 Call validate_password(password),

 If invalid,

 Call handle_encryption_error,

 Return Generate random salt,

 Call derive_key_from_password(password, salt),

 Encrypt plaintext using AES-256 with derived key,

 Concatenate salt with encrypted data,

 Return concatenated ciphertext

+decrypt_note(ciphertext, password): SDD_TT_04_006 :: [SRD::T_16, SRD::T_28,
SRD::T_36, SRD::T_46, SRD::T_50]

```
Call validate_password(password),  
If invalid, Call handle_decryption_error,  
    Return Extract salt from ciphertext,  
Call derive_key_from_password(password, salt),  
Decrypt ciphertext using AES-256 with derived key,  
Return plaintext
```

4.2.3 DBConnect Class

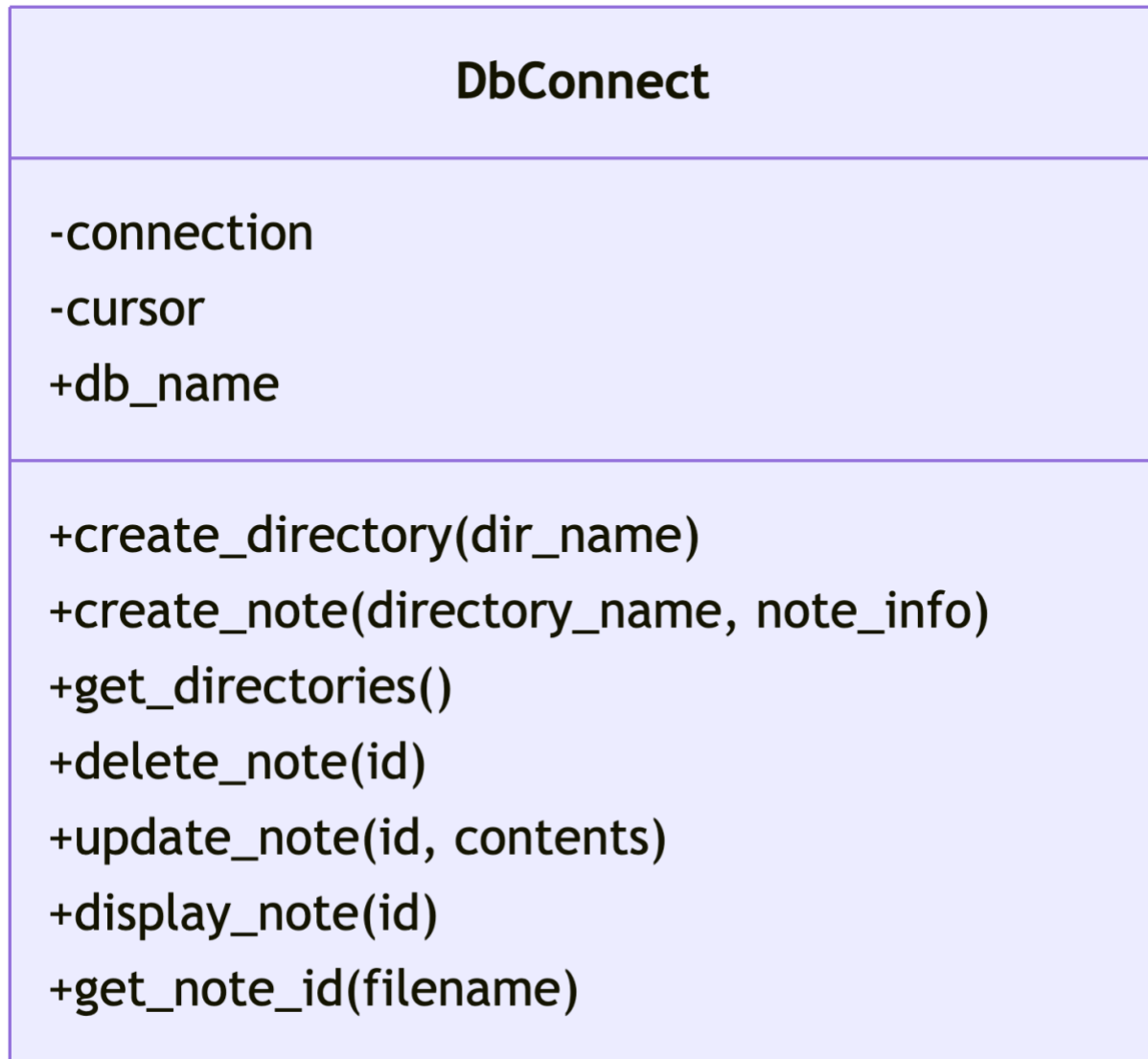


Figure7: DBConnect Class

The database class is a relatively straightforward database connector that utilizes the Python library “sqlite3” to perform rudimentary tasks on a local database file on the user’s computer. The database class supports create, read, update, and delete operations. Only one design change is notable, and that was the decision between hard deletion and soft deletion.

While normal SQL practices would recommend soft deletion, which is setting a record to inactive in contrast from removing it from the database, we ultimately chose soft deletion, the ladder option. This choice would prioritize security for the user, so that when a note is deleted, the user can be certain there is no chance in recovering the contents of that note. The database is also only accessible through private methods, ensuring protection from injection attacks and other SQL specific security threats. The combination of these design decisions, with assistance from the security manager class, ensures the users' notes are safe on their local device.

idx	filename	contents
0	file1.txt	This is my first note!
1	file2.txt	This is the contents of my second note

Figure 8: DB Schema

The schema, outline of the database, outlined in figure 6, is simple, having an index, filename, and contents columns. The index is used for internal reference; the filename is what is displayed to the user; and the contents hold the body of the respective note. The software also allows the user to create a single layered directory for each note, represented by tables in the database itself.

4.2.3.1 DBConnect Pseudocode:

```
-create_note(directory_name, note_info): SDD_TT_1_001
    find_current_date()
    insert (note_info) into (directory_name)
    return new_note (confirmation) or None (fail)
```

```
-delete_note(note_id, directory_name): SDD_TT_1_002
{note: this comes after the menu asks for confirmation. there is no safety for this method.}
    select note_id
    confirm query length == 1
    hard delete note
    return 0 if success, 1 is error
```

```
-display_note(note_id, directory_name): SDD_TT_1_003
    select note_id
    return note
```

```
-update_note(note_id, directory_name, updated_contents): SDD_TT_1_004
    update (directory_name)
    set contents = updated_contents
    select * from (directory_name) where note_id
    return ^^ for confirmation
```

```
-get_note_id(directory_name, filename): SDD_TT_1_005
    select note_id from directory_name where = filename
    return ^^
```

4.2.3.2 Future Updates Pseudocode

```
-create_directory(directory_name):
    create table (dname) with values
    select table
    if table_name matches directory_name,
        return 0
    else,
        return 1
```

```
-get_directories(None):
    select all tables from master sheet
    return list of tables
```

4.2.4 AI Class

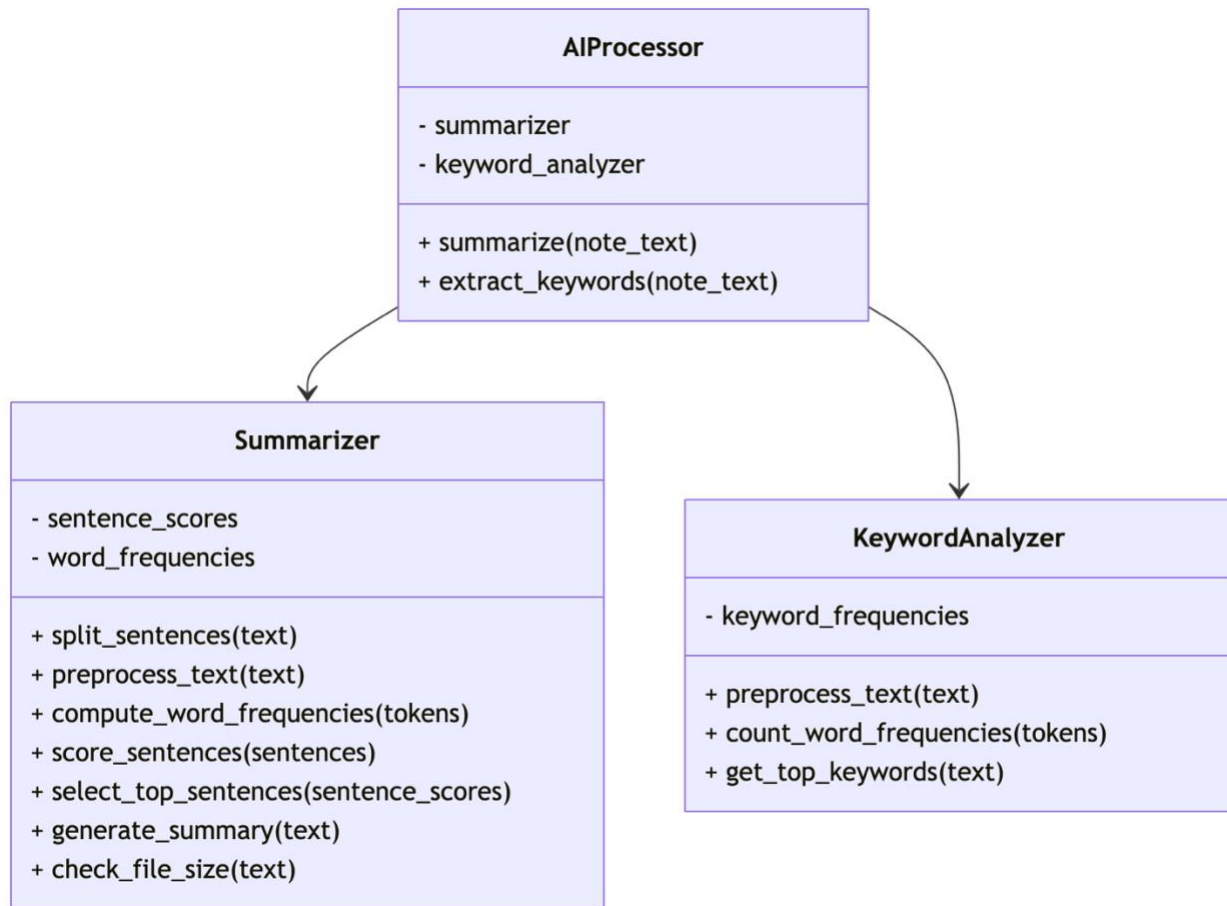


Figure 9: AI Class Diagram

4.2.4 AIProcessor Class

Figure 9 shows the class structure of the AI processing portion of the Secure Notes Application. The **AIProcessor** class serves as the entry point for AI-related operations. It does not perform text analysis directly. Instead, it stores references to two specialized analysis classes, **Summarizer** and **KeywordAnalyzer**, and delegates the requested operation to the appropriate class.

The **AIProcessor** class contains two attributes:

- **summarizer**, which stores the **Summarizer** object used for note summarization
- **keyword_analyzer**, which stores the **KeywordAnalyzer** object used for keyword extraction

The class defines two public methods:

- **summarize(note_text)**, which receives decrypted note text and forwards it to the **Summarizer** class
- **extract_keywords(note_text)**, which receives decrypted note text and forwards it to the **KeywordAnalyzer** class

This design keeps AI task coordination separate from the implementation of the analysis algorithms.

4.2.4.1 Summarizer Subclass

The **Summarizer** class implements the system's extractive summarization algorithm. It receives decrypted note text and processes it through several stages to produce a concise summary.

The class contains two main attributes:

- **sentence_scores**, which stores the score assigned to each sentence during analysis
- **word_frequencies**, which stores the frequency of significant words found in the note text

The Summarizer class defines the following methods:

- **split_sentences(text)**, which separates the note into individual sentences and preserves their original order
- **preprocess_text(text)**, which normalizes the input text by converting it to lowercase, removing punctuation, removing stopwords, and tokenizing words
- **compute_word_frequencies(tokens)**, which counts the occurrences of significant words in the note and stores them in the word_frequencies structure
- **score_sentences(sentences)**, which calculates a score for each sentence based on the frequencies of the meaningful words it contains
- **select_top_sentences(sentence_scores)**, which identifies the highest-ranked sentences to include in the final summary
- **generate_summary(text)**, which coordinates the full summarization process and returns the final summary text
- **check_file_size(text)**, which verifies that the input text remains within supported size limits before analysis begins

The Summarizer class returns a summary composed of the highest-ranked sentences arranged in their original order so that the output remains readable and consistent with the original note.

4.2.4.2 KeywordAnalyzer Subclass

The **KeywordAnalyzer** class implements the system's keyword extraction functionality. It receives decrypted note text and identifies the most significant words based on their frequency in the note.

The class contains one main attribute:

- **keyword_frequencies**, which stores the frequency count of relevant words identified during processing

The KeywordAnalyzer class defines the following methods:

- **preprocess_text(text)**, which normalizes the input text by converting it to lowercase, removing punctuation, removing stopwords, and tokenizing words
- **count_word_frequencies(tokens)**, which computes the frequency of each meaningful token in the note text and stores the results in the keyword_frequencies structure
- **get_top_keywords(text)**, which coordinates the keyword extraction process and returns a ranked list of the most significant words

The KeywordAnalyzer class returns a list of keywords that the AIPProcessor passes back to the Menu class for display to the user.

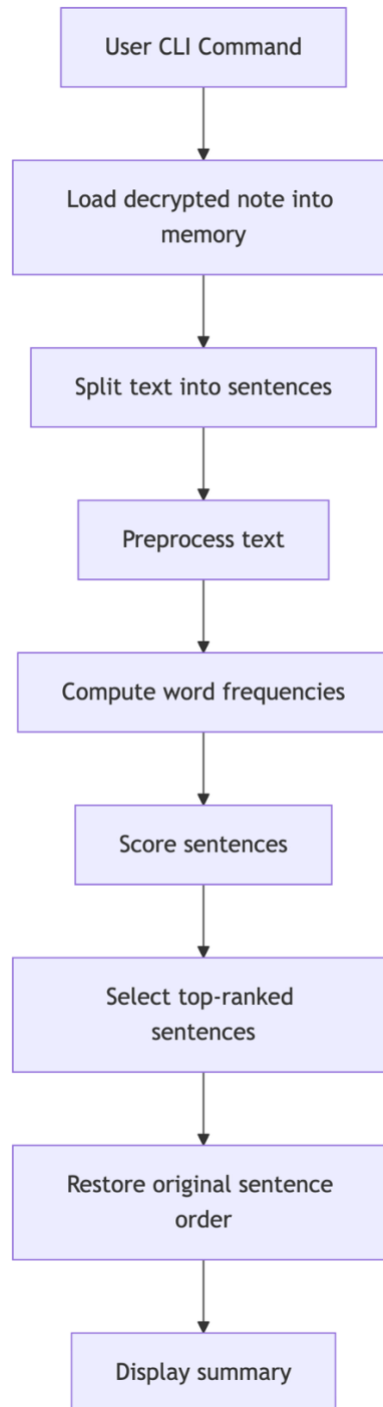


Figure 10: AI Sequence Diagram

Figure 10 shows the system's processing pipeline for generating an extractive summary of a note. The process begins when the user requests a summary through the CLI. The system loads the decrypted note into memory and passes the text to the summarization module.

The summarization algorithm first splits the note into individual sentences. It then preprocesses the text by normalizing words and removing unnecessary elements. After preprocessing, the

system computes word frequencies and uses those frequencies to score each sentence based on its informational importance.

Next, the algorithm selects the highest-ranked sentences to form the summary. It restores the sentences to their original order to preserve readability and context. Finally, the system displays the generated summary to the user.

4.2.4.3 AI Pseudocode:

```
-summarize(note_text): SDD_TT_2_001 :: [SRD::T_12]
    If note_text is empty,
        Return error
    summary = generate_summary(note_text),
    Return summary

-extract_keywords(note_text): SDD_TT_2_002 :: [SRD::T_18]
    If note_text is empty,
        Return error
    keywords = get_top_keywords(note_text),
    Return keywords

-generate_summary(text): SDD_TT_2_003 :: [SRD::T_12]
    sentences = split_sentences(text),
    If number of sentences == 0,
        Return error
    tokens = preprocess_text(text),
    If number of tokens == 0,
        Return error
    word_frequencies = compute_word_frequencies(tokens),
    sentence_scores = score_sentences(sentences),
    top_sentences = select_top_sentences(sentence_scores),
    Sort top_sentences by original sentence order,
    summary = join top_sentences into one text,
    Return summary

-get_top_keywords(text): SDD_TT_2_004 :: [SRD::T_18]
    tokens = preprocess_text(text),
    If number of tokens == 0,
        Return error
    keyword_frequencies = count_word_frequencies(tokens),
    keywords = select top keywords from keyword_frequencies,
    Return keywords

-check_file_size(text): SDD_TT_2_005 :: [SRD::T_24]
    if sizeof(text) > MAX_SIZE,
        print("File exceeded maximum size")
        Return 0
    Else,
```

Return 1

5 Trace Tags

A compiled list of the trace tags in the SDD, and how they link to the SRD.

Note: Each bullet point represents a trace tag, with the first level being HLD class trace tags, the second level being HLD responsibility trace tags, the third level being LLD member function trace tags, and the final level being the SRD trace tags.

For example, SRD::T_02 is fulfilled by the Menu class, namely the first objective of the menu class, by the interaction of the low- level trace tag functions associated with that HLD tag..

- **Menu Trace Tags: [SDD_HLD_01_MENU]**

- SDD_HLD_MENU_001
 - SDD_TT_3_002
 - SDD_TT_3_003
 - SDD_TT_3_006
 - SDD_TT_3_010
 - SDD_TT_3_014
 - SRD::T_02
 - SRD::T_03
 - SRD::T_04
 - SRD::T_11
 - SRD::T_12
 - SRD::T_13
 - SRD::T_14
 - SRD::T_21
 - SRD::T_22
 - SRD::T_29
 - SRD::T_30
 - SRD::T_39
 - SRD::T_44
 - SRD::T_45
- SDD_HLD_MENU_002
 - SDD_TT_3_001
 - SDD_TT_3_005
 - SDD_TT_3_007
 - SDD_TT_3_008

- SDD_TT_3_009
- SDD_TT_3_012
 - SRD::T_06
 - SRD::T_11
 - SRD::T_12
 - SRD::T_13
 - SRD::T_14
 - SRD::T_19
 - SRD::T_23
 - SRD::T_53
 - SRD::T_54
- SDD_HLD_MENU_003
 - SDD_TT_3_003
 - SDD_TT_3_004
 - SDD_TT_3_010
 - SDD_TT_3_011
 - SRD::T_02
 - SRD::T_04
 - SRD::T_12
 - SRD::T_13
 - SRD::T_14
 - SRD::T_20
 - SRD::T_28
 - SRD::T_29
 - SRD::T_31
 - SRD::T_39
 - SRD::T_43
- **SecurityManager Trace Tags: [SDD_HLD_02_SEC]**
 - SDD_HLD_SEC_001
 - SDD_TT_4_001
 - SDD_TT_4_002
 - SRD::T_15
 - SRD::T_31
 - SRD::T_46
 - SDD_HLD_SEC_02
 - SDD_TT_4_003
 - SDD_TT_4_005

- SRD::T_15
 - SRD::T_28
 - SRD::T_35
 - SRD::T_46
 - SRD::T_47
 - SRD::T_49
 - SRD::T_51
- SDD_HLD_SEC_003
 - SDD_TT_4_004
 - SDD_TT_4_006
 - SRD::T_16
 - SRD::T_28
 - SRD::T_36
 - SRD::T_46
 - SRD::T_50
 - SRD::T_52
- **DBConnect Trace Tags: [SDD_HLD_03_DB]**
 - SDD_HLD_DB_001
 - SDD_TT_1_001
 - SDD_TT_1_002
 - SDD_TT_1_003
 - SDD_TT_1_004
 - SDD_TT_1_005
 - SRD::T_11
 - SRD::T_13
 - SRD::T_14
 - SRD::T_19
 - SRD::T_23
 - SRD::T_27
 - SRD::T_41
 - SRD::T_42
 - SRD::T_48
- **AI Trace Tags: [SDD_HLD_04_AI], [SDD_HLD_04_01_SUM], [SDD_HLD_04_01_KEY]**
 - SDD_HLD_AI_001
 - SDD_TT_2_001
 - SDD_TT_2_005

SE 300



- SRD::T_12
 - SRD::T_24
 - SDD_HLD_SUM_001
 - SDD_TT_2_003
 - SRD::T_12
 - SDD_HLD_KEY_001
 - SDD_TT_2_002
 - SDD_TT_2_004
 - SRD::T_18
-

Appendix and Review Driven Modifications

1. Single App-Level Password
 - a. Implemented one master password collected once when the app starts rather than asking for a password on every single note operation. User types it in at launch, it gets validated, and then it gets passed around to encrypt and decrypt as needed throughout the session. Satisfies T_28 and T_31 without being annoying or repetitive to use.
2. Plaintext Stays in Memory
 - a. No temp files anywhere. After we decrypt a note the plaintext just lives as a variable, gets used (displayed, appended to, or summarized), and disappears when the function ends. This was a fix from the original Menu pseudocode which was writing to a temp file on disk. That violated T_05 and T_50 so we modified it.
 - b. Note: Upon review between John and TJ, we resolved this issue as a difference of definitions. Read *Menu – Temporary Files* for resolution.
3. SecurityManager Is Stateless
 - a. Took out key, salt, and **init** from SecurityManager. Key and salt are just local variables inside encrypt_note and decrypt_note now -- they pop into existence, do their thing, and are gone. No reason to store them on the object and honestly its more secure this way.
4. SecurityManager Public Interface
 - a. Only encrypt_note() and decrypt_note() are now public. Everything else (derive_key_from_password, validate_password, handle_encryption_error, handle_decryption_error) is private. If you need to encrypt or decrypt something, you call one of those two methods.
5. Single App-Level Password variable and function in Menu Class
 - a. Why? This fix resolves the password issues raised in Review #1
 - b. Add private password variable to Menu class
 - c. Add public member get_password() that prompts the user for a password and saves it to the password variable.
6. Temporary Files
 - a. This fix resolves the issue raised in Review #2
 - b. Throughout the menu class, I used the syntax of passing and writing temporary files. These files exist nowhere in persistent memory. Noting the difference of definitions between John and I, I have since added a definition of temporary file to the document. This clarification ensures that the SDD doesn't violate SRD::T_05 or SRD::T_50.
7. Persistent Password in DB
 - a. After reviewing the concerns of John with regards to the Menu class and password, I added functions to the menu class to allow the user. to create a persistent password that saves to the DB.
 - b. After further review with all members, we decided not to use a persistent password. So, I removed the persistent password methods from the Menu class.

8. Mapping of SRD Trace Tags to SDD functions
 - a. After John noted that SRD::T_21 wasn't mapped to any functions, I included it in the trace tags for SDD_TT_03_014, resolving this issue.
 - b. SRD::T_24 and SRD::T_25 were unmapped. Now, they are traced to a function in the AI class that checks file size before summarizing.
9. Trace Tag Discrepancy
 - a. Upon reviewing the SRD, I noticed that the trace tags T_12 and T_17 are the same. So, we mapped the functions only to T_12.
10. DB member functions pseudocode
 - a. Resolve mapping issue between class diagram and pseudocode for create_note function
 - b. Add pseudocode to append note function
 - c. Add pseudocode for log access function, ensuring the log never includes plaintext or passwords
11. Trace Tag Refactor
 - a. Upon reviewing the SDD with Professor King, he noted that the overall flow and connection of the trace tags was off
 - b. Instead of mapping each function directly to an SRD requirement trace tag, we needed to map each function- a LLD trace tag- to a HLD trace tag.
 - c. Finally, we connect the HLD tags to the SRD ones
 - d. So, we refactored the trace tags

Other Review Considerations: If a user requests to summarize a note that has exceeded the maximum size, then the original note will be displayed.